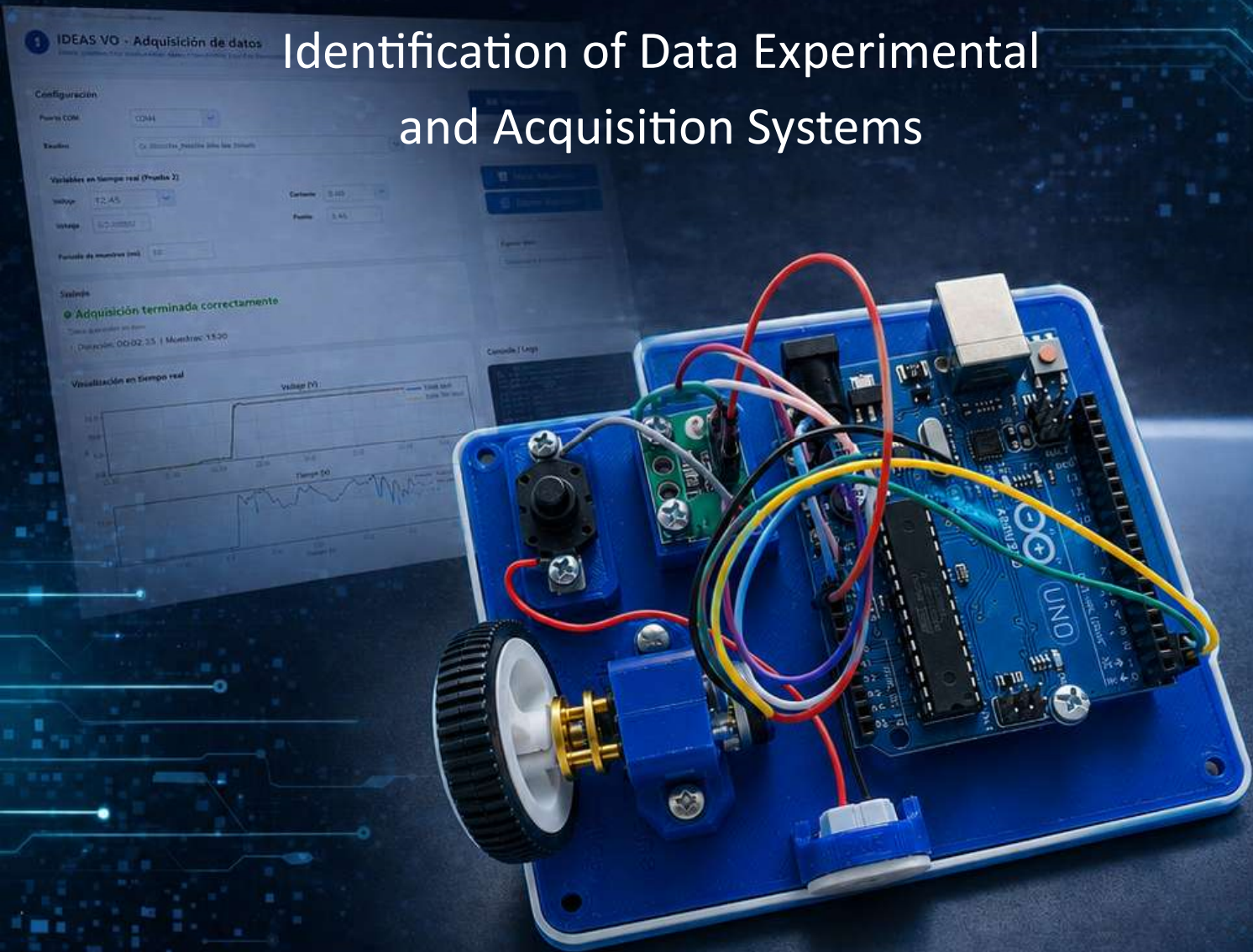


IDEAS VO

ADQUISICIÓN DE DATOS

Identification of Data Experimental
and Acquisition Systems



MANUAL DE USUARIO

Práctica 1. Adquisición de datos con el kit IDEAS v0

Objetivo de la práctica

El objetivo de esta práctica es aprender a **adquirir datos experimentales** usando el kit de adquisición de datos **IDEAS v0**. En particular, se registrarán dos variables importantes del funcionamiento de un motor de corriente directa:

Corriente eléctrica, medida en miliamperes, que nos indica cuánta energía está consumiendo el motor durante su operación.

Velocidad de giro, expresada mediante pulsos del encoder, que posteriormente nos permitirá calcular las **RPM**, es decir, las revoluciones por minuto del motor.

Además, los datos obtenidos serán visualizados mediante gráficas usando una interfaz desarrollada en **Python**, lo que permitirá observar el comportamiento del motor en tiempo real.

Esta práctica es una introducción al uso de sensores, adquisición de datos, comunicación serial y análisis básico de sistemas físicos mediante herramientas de programación.

Materiales incluidos en el kit IDEAS v0

Para realizar la práctica se utilizarán los siguientes componentes:

Tarjeta Arduino UNO con cable USB

Se encarga de leer las señales del encoder y del sensor de corriente, además de enviar los datos a la computadora mediante comunicación serial.

Motor N20 con encoder y rueda

Es el sistema físico que se va a analizar. El motor gira cuando recibe alimentación eléctrica y el encoder permite medir su movimiento.

Sensor de corriente INA240

Permite medir la corriente que consume el motor durante su funcionamiento.

Botón de encendido/apagado

Se utiliza para controlar manualmente cuándo se enciende o se apaga el motor.

Conector USB tipo C

Sirve para alimentar el motor con una fuente externa, por ejemplo, un cargador de celular.

Cables Dupont para conexión

Se utilizan para realizar las conexiones eléctricas entre los diferentes componentes del sistema.

Desarrollo de la práctica

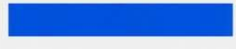
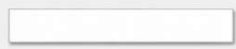
1. Verificación de conexiones

Antes de iniciar la práctica, es importante revisar cuidadosamente que todos los componentes estén correctamente conectados.

Se debe verificar la conexión del **motor DC N20 con encoder**, la **fuentes de voltaje**, el **sensor de corriente INA240**, la **tarjeta Arduino UNO**, el **botón de encendido/apagado** y la **conexión USB hacia la computadora**.

Esta revisión es muy importante porque una conexión incorrecta puede provocar lecturas erróneas, que el motor no funcione o incluso que algún componente se dañe.

Método de cableado del encoder:

	Rojo: alimentación del motor +5V viene de IN- del sensor INA240
	Negro: alimentación del encoder +3.3V usando el voltaje del Arduino (los polos positivo y negativo deben conectarse incorrectamente)
	Amarillo: retroalimentación de canal 2 del encoder (C2 del motor conectado a pin digital 2 del Arduino)
	Verde: retroalimentación de canal 1 del encoder (C1 del motor conectado a pin digital 3 del Arduino)
	Azul: alimentación del encoder - conectado a tierra común
	Blanco: alimentación del motor - conectado a tierra común



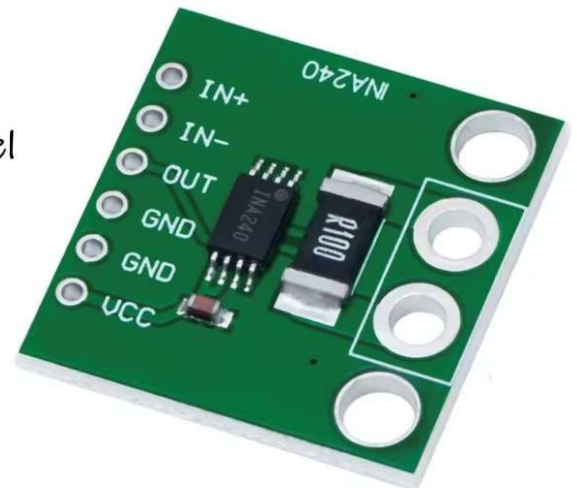
IN+ : entrada de voltaje 5v desde el botón de encendido/apagado

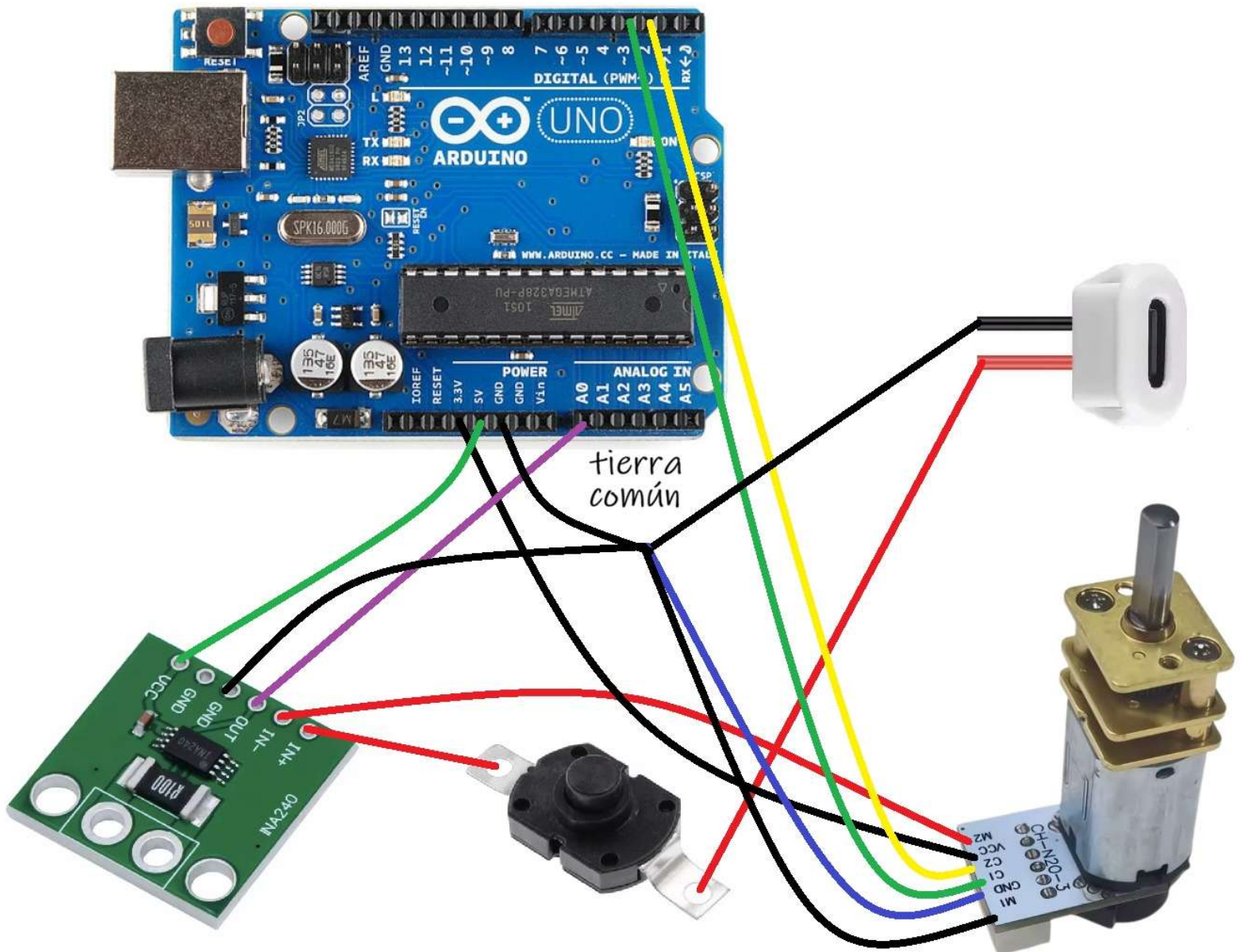
IN- : salida de voltaje hacia la alimentación del motor n20

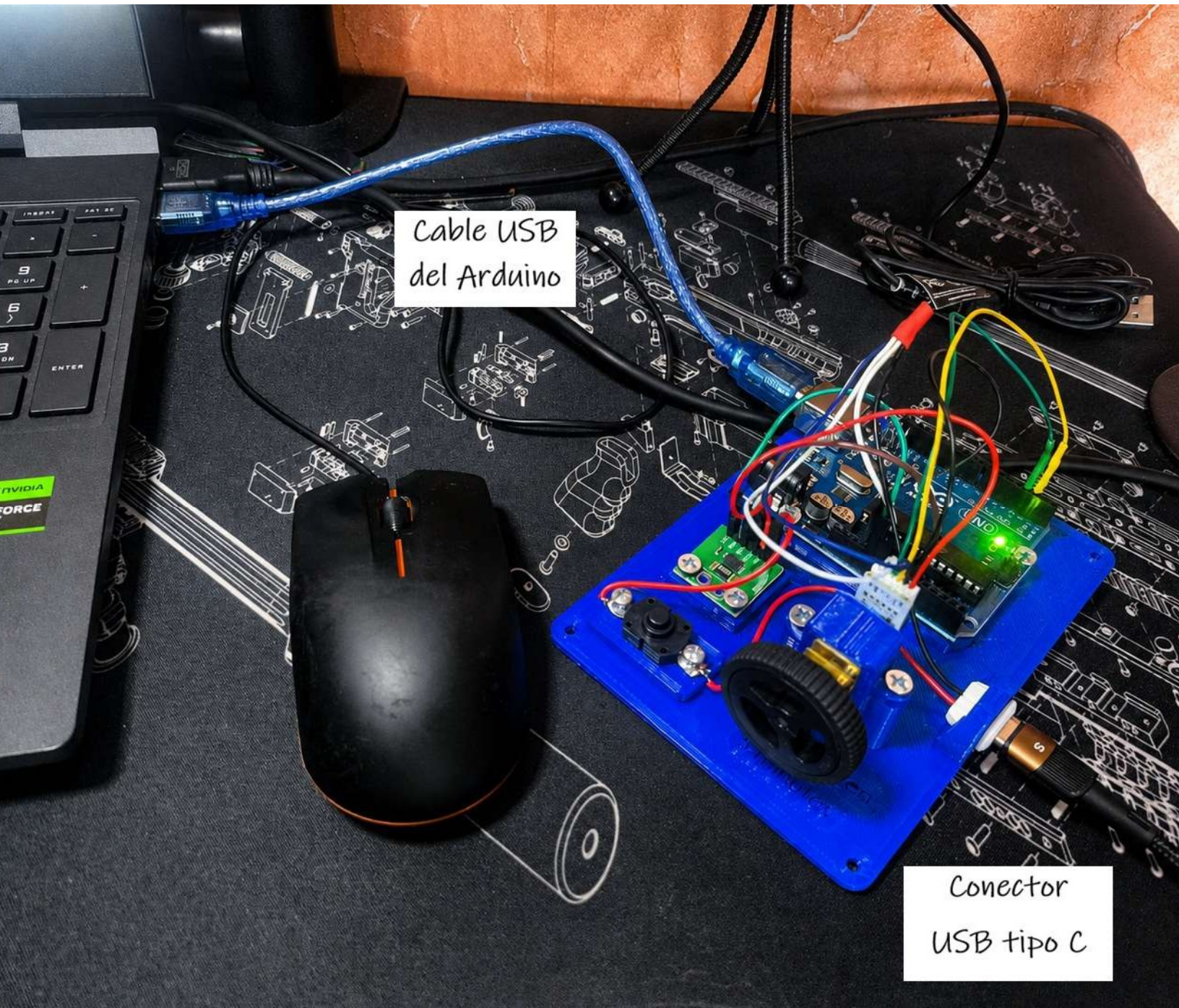
OUT: salida de datos hacia A0 del arduino

GND: conectado a tierra común

VCC: alimentación del sensor +5v usando el voltaje del arduino







Cable USB
del Arduino

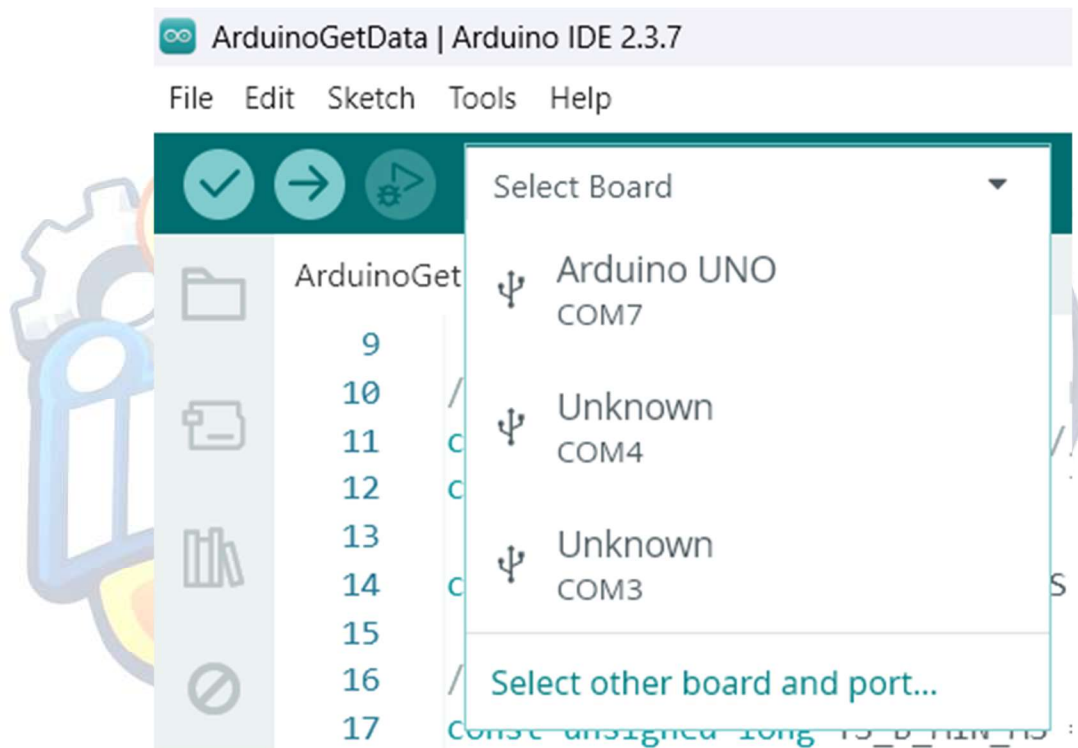
Conector
USB tipo C

2. Selección de la tarjeta Arduino

Abrimos el **Arduino IDE** y revisamos en la opción “**Select Board**” el puerto en el que la computadora reconoce la tarjeta Arduino.

En este ejemplo, la tarjeta fue reconocida en el puerto: **COM7**

El puerto puede cambiar dependiendo de la computadora, por lo que cada estudiante debe verificar cuál aparece en su equipo.



3. Carga del programa en Arduino

Abrimos el archivo llamado:

ArduinoGetData

Después, cargamos el programa en la tarjeta Arduino presionando el botón de carga,



representado por una flecha en el Arduino IDE.

Este programa permitirá que Arduino lea los datos del sensor de corriente y del encoder, y posteriormente los envíe a la computadora.

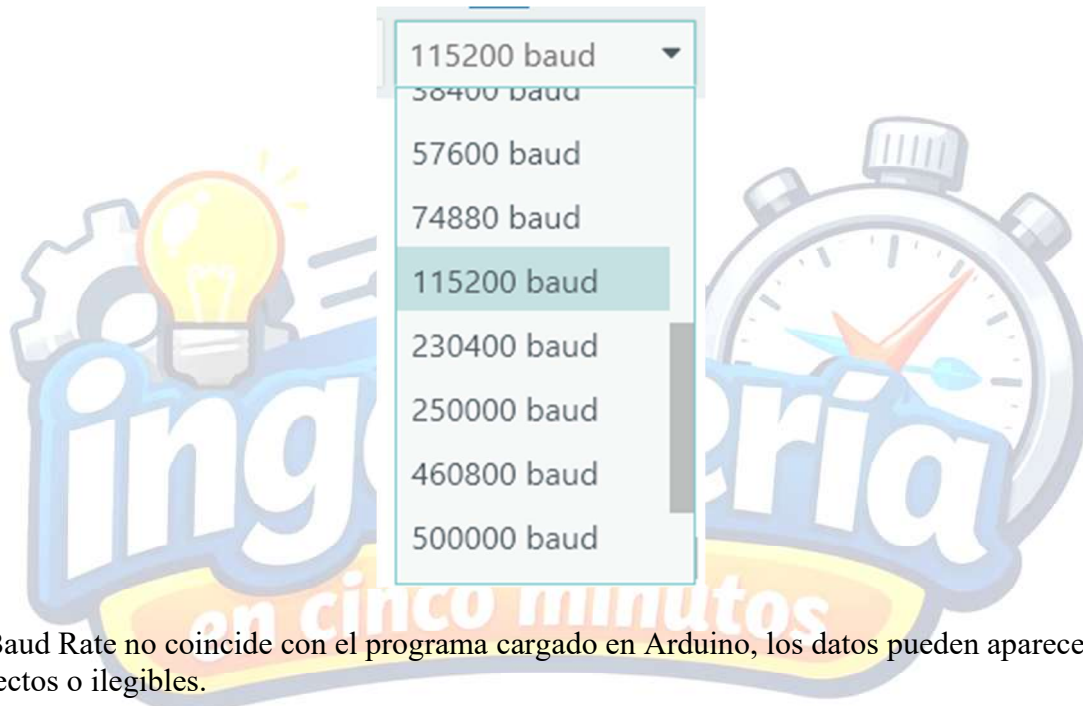
4. Revisión del monitor serial

Antes de conectar la alimentación del motor mediante el conector USB tipo C, abrimos el

Monitor Serial desde el Arduino IDE.



Al abrirlo, debemos asegurarnos de que la velocidad de comunicación, conocida como **Baud Rate**, esté configurada en: **115200**



Si el Baud Rate no coincide con el programa cargado en Arduino, los datos pueden aparecer incorrectos o ilegibles.

En este punto, el sistema debe mostrar un mensaje inicial indicando que el programa está listo para comenzar la lectura de datos.

```
Calibrando INA240...  
Deja el motor sin alimentacion.  
VZERO=2.5194
```

5. Encendido del motor y lectura de datos

Ahora conectamos la alimentación mediante el **conector USB tipo C**.

El botón del kit permite controlar el encendido y apagado del motor. Cuando el motor esté encendido, en el Monitor Serial deberán aparecer valores similares a los esperados para la práctica.

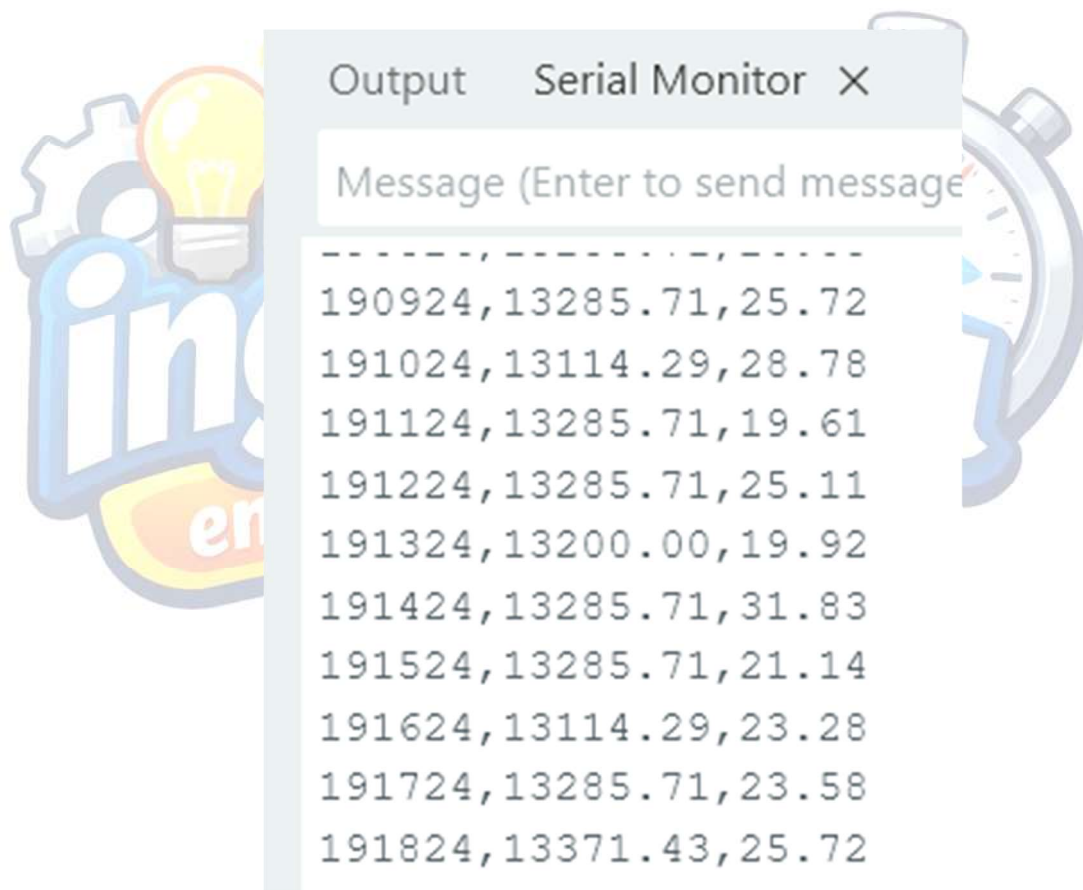
Los datos mostrados corresponden a:

Tiempo en milisegundos, que indica el instante en el que se tomó cada medición.

Pulsos del encoder, que representan el movimiento del motor. En este caso, aproximadamente **7 pulsos equivalen a una revolución completa del motor**, es decir, una vuelta.

Corriente en miliamperes, que indica el consumo eléctrico del motor durante su funcionamiento.

Estos datos son importantes porque permiten analizar cómo responde el motor cuando se enciende, cómo cambia su velocidad y cuánta corriente consume.



6. Ejecución del programa en Python

Después de verificar que Arduino está enviando datos correctamente, cerramos el Arduino IDE para evitar conflictos con el puerto serial.

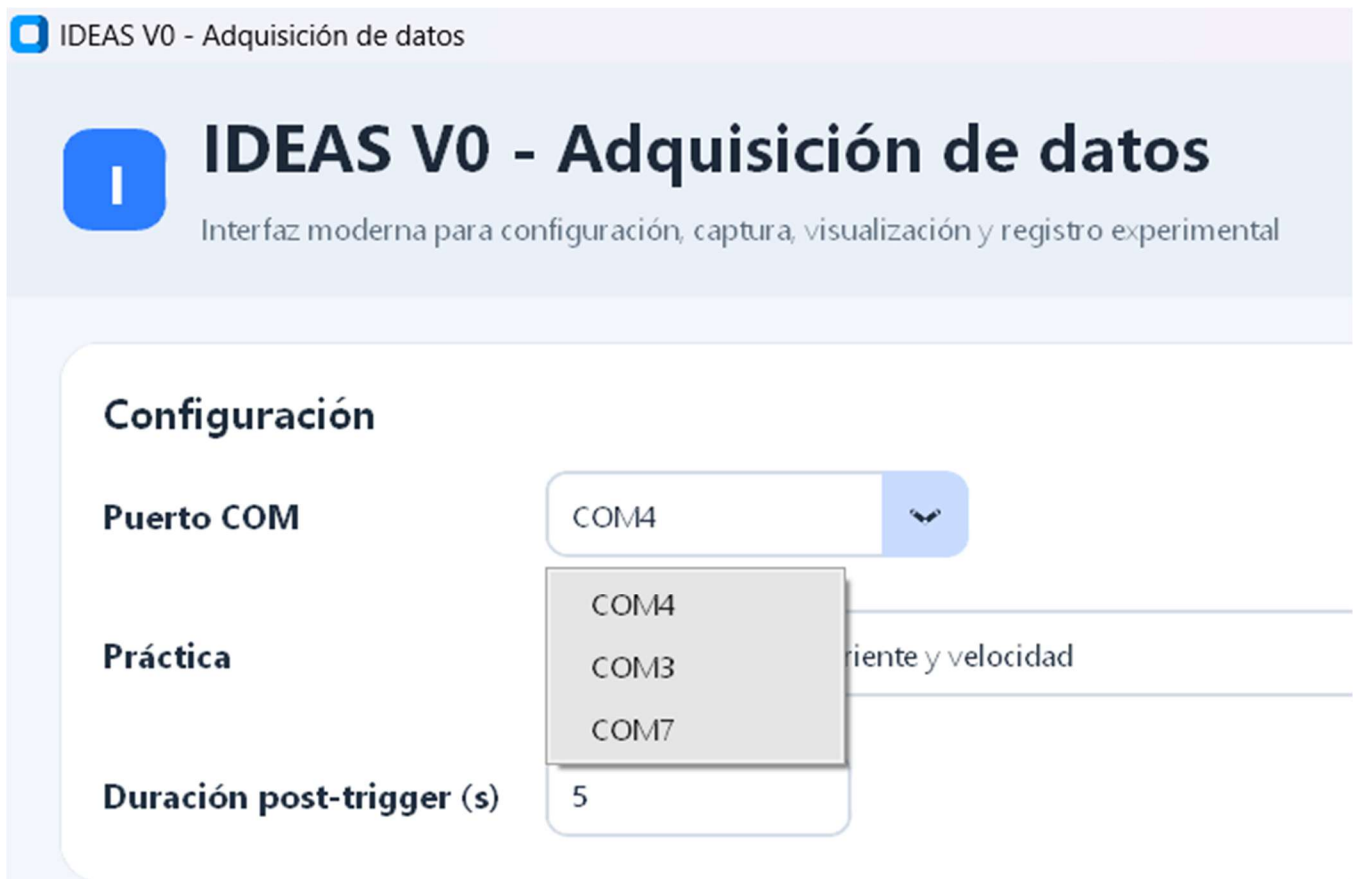
Luego, abrimos **PyCharm** y creamos un nuevo proyecto “IdentificacionSistemas”. Copiamos el archivo: **practical.py**

en la siguiente ruta de ejemplo:

```
C:\Users\tuUsuario\PycharmProjects\IdentificacionSistemas\.venv\Scripts
```

Después ejecutamos el archivo. Al correr el programa, se abrirá una pantalla o interfaz gráfica en Python.

En esta interfaz debemos seleccionar el puerto COM en el que la computadora reconoció la tarjeta Arduino. En este ejemplo se utilizó: **COM7**



7. Selección de la práctica

Dentro de la interfaz, seleccionamos la opción correspondiente a: **Práctica 1**

Esto le indica al programa que realizará la adquisición de datos del motor, leyendo las señales de corriente y encoder.

Práctica

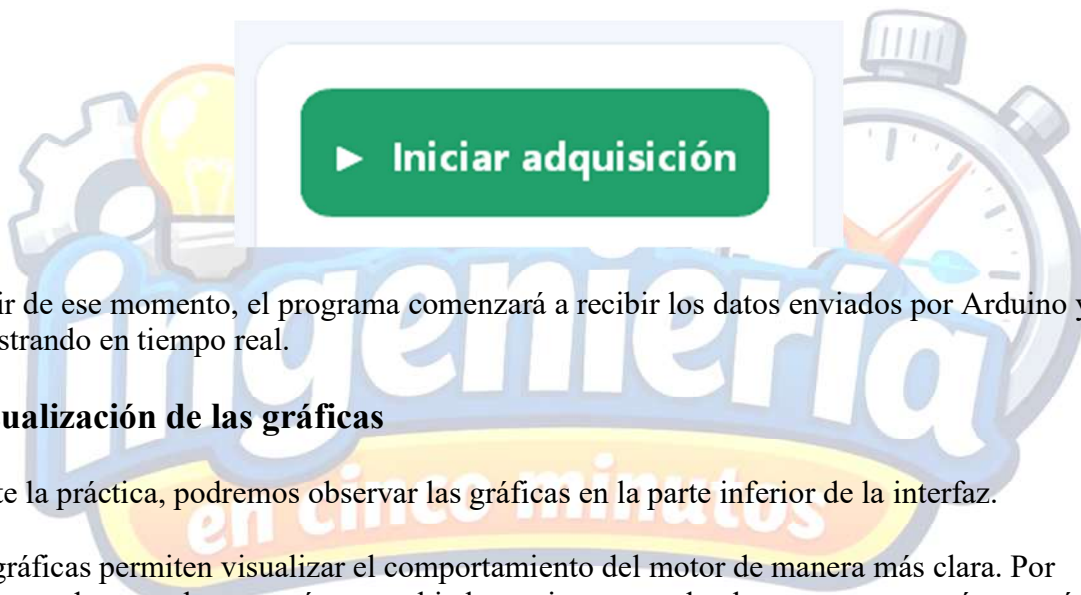
1 - Monitoreo de corriente y velocidad

Duración post-trigger (s)

- 1 - Monitoreo de corriente y velocidad
- 2 - Efecto de perturbaciones mecánicas
- 3 - Filtrado y repetibilidad de señales
- 4 - Evaluación de tasa de muestreo

8. Inicio de la adquisición de datos

Para comenzar la adquisición, presionamos el botón correspondiente a **iniciar adquisición**.



A partir de ese momento, el programa comenzará a recibir los datos enviados por Arduino y los irá mostrando en tiempo real.

9. Visualización de las gráficas

Durante la práctica, podremos observar las gráficas en la parte inferior de la interfaz.

Estas gráficas permiten visualizar el comportamiento del motor de manera más clara. Por ejemplo, podremos observar cómo cambia la corriente cuando el motor arranca, cómo varían los pulsos del encoder y cómo se comporta el sistema durante el encendido y apagado.

Observar los datos en forma de gráfica facilita mucho el análisis, ya que no solo vemos números, sino también el comportamiento del sistema a través del tiempo.

Visualización en tiempo real



10. Generación de archivos de datos

Al finalizar la práctica, el programa genera archivos con los datos adquiridos.

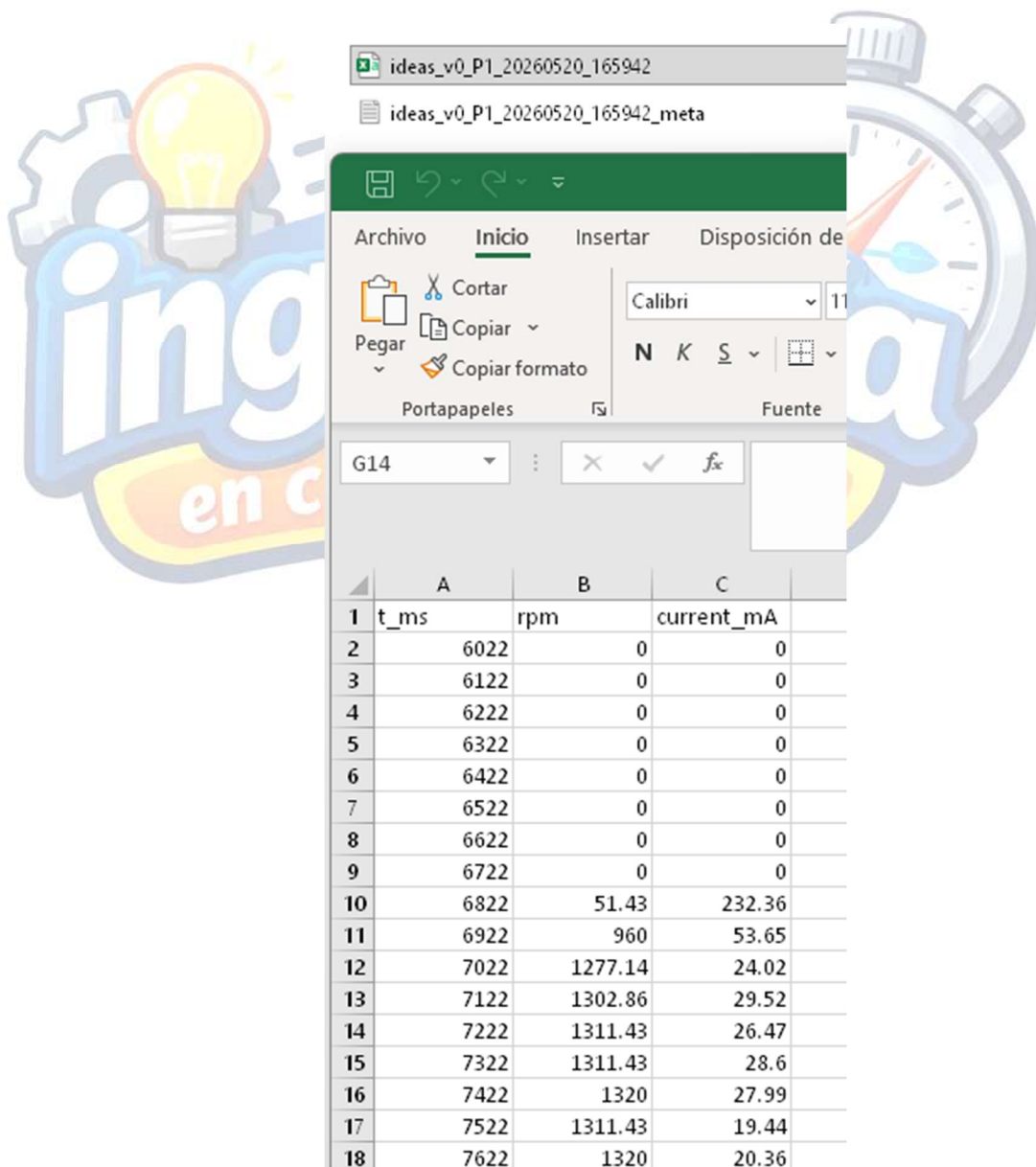
Estos archivos se guardan dentro de la carpeta **datos** del proyecto de PyCharm. Una ruta de ejemplo puede ser:

```
C:\Users\tuUsuario\PycharmProjects\IdentificacionSistemas\.venv\Scripts\datos
```

Los datos se almacenan en formato: `.csv`

Este tipo de archivo puede abrirse en programas como Excel, Google Sheets o también puede analizarse usando Python.

Los archivos generados serán útiles para prácticas posteriores, donde se realizará un análisis más detallado del motor.



The screenshot shows a CSV file named 'ideas_v0_P1_20260520_165942' opened in Excel. The file is displayed in a table with 3 columns: 't_ms', 'rpm', and 'current_mA'. The data is organized into 18 rows. The first 9 rows show 'rpm' and 'current_mA' as 0, while the last 9 rows show varying values. The background of the image features a watermark with a lightbulb and the text 'Ingeniería en 5 minutos'.

	A	B	C
1	t_ms	rpm	current_mA
2	6022	0	0
3	6122	0	0
4	6222	0	0
5	6322	0	0
6	6422	0	0
7	6522	0	0
8	6622	0	0
9	6722	0	0
10	6822	51.43	232.36
11	6922	960	53.65
12	7022	1277.14	24.02
13	7122	1302.86	29.52
14	7222	1311.43	26.47
15	7322	1311.43	28.6
16	7422	1320	27.99
17	7522	1311.43	19.44
18	7622	1320	20.36

Conclusiones

En esta práctica se logró adquirir información experimental del funcionamiento de un motor DC usando el kit **IDEAS v0**.

Se obtuvieron datos de **tiempo en milisegundos, pulsos del encoder y corriente en miliamperes**. Estos datos representan el comportamiento real del sistema y son el primer paso para aprender cómo se analizan dispositivos físicos mediante sensores, electrónica y programación.

En prácticas posteriores, estos datos podrán utilizarse para calcular la velocidad del motor en **RPM**, graficar con mayor detalle su comportamiento e iniciar el proceso de **identificación de sistemas**, que consiste en obtener un modelo matemático a partir de datos reales.

Esta práctica marca el inicio de una serie de actividades enfocadas en aprender conceptos básicos de ingeniería mediante proyectos reales. La idea es que puedas conectar la teoría con la práctica, observar el comportamiento de un sistema físico y usar herramientas modernas como Arduino y Python para analizarlo.

Sigue practicando, experimentando y aprendiendo.

Saludos. Síguenos en redes sociales como Ingeniería en 5 minutos.